

Ollama Masterclass: 5-Part Study Guide

Interview Preparation Notes

Contents

1	Part 1: LLM Fundamentals & The Ollama Ecosystem	2
1.1	Proprietary vs. Open Source Models	2
1.2	The Problem Ollama Solves	2
1.3	Key Benefits of Local Execution	2
1.4	Code Snippet: Verification	2
2	Part 2: Managing and Running Models via CLI	2
2.1	Core CLI Commands	2
2.2	In-Session (Slash) Commands	3
2.3	Multimodal Execution (Vision)	3
3	Part 3: Interacting via the Python Library	4
3.1	Installation and Basic Generation	4
3.2	Customizing Generation Parameters	4
3.3	Maintaining Context with the Chat Method	4
3.4	Multimodal Input (Vision via Python)	5
4	Part 4: Advanced Capabilities - Tool Calling & Modelfiles	5
4.1	Tool Calling (Function Calling)	5
4.1.1	The 4-Step Tool Calling Workflow	5
4.1.2	Python Implementation Example	6
4.2	Custom Models using Modelfiles	7
4.2.1	Key Modelfile Directives	7
4.2.2	Building a Custom Sentiment Analysis Model	7
5	Part 5: Architecture, LangChain & Ollama Cloud	8
5.1	The Underlying REST API	8
5.2	Orchestration with LangChain	8
5.2.1	LangChain Code Example	8
5.3	Bypassing Hardware Limits with Ollama Cloud	9
5.3.1	Cloud Workflow	9

1 Part 1: LLM Fundamentals & The Ollama Ecosystem

1.1 Proprietary vs. Open Source Models

- **Proprietary Models:** Owned by large corporations (e.g., OpenAI, Google). They require API keys, internet access, and subscription fees. Architecture and weights remain hidden on company servers.
- **Open Source Models:** Openly available (e.g., Meta Llama, DeepSeek, Qwen). Users can download the raw weights and architecture to run or fine-tune them locally.

1.2 The Problem Ollama Solves

Running raw open-source models natively is highly complex. It requires optimizing hardware, managing system RAM vs. VRAM, and resolving dependency issues. **Ollama** acts as a wrapper that entirely removes this friction. It manages storage, handles optimization, and allocates memory automatically behind the scenes.

1.3 Key Benefits of Local Execution

- **Total Privacy:** Your data (prompts and documents) never leaves your local machine.
- **Offline Access:** Once the model is pulled, inference requires zero internet connection.
- **Predictable Cost:** 100% free local execution without API token limits.

1.4 Code Snippet: Verification

Once installed, you can verify Ollama is active on your system using a simple terminal command:

```
ollama --version
```

2 Part 2: Managing and Running Models via CLI

2.1 Core CLI Commands

The command line is the primary environment for experimenting with models.

- **Download a Model:** Fetches the model weights from the Ollama registry to your local storage without running it.

```
ollama pull llama3.2:1b
```

- **List Local Models:** Displays all models currently stored on your machine, along with their file sizes.

```
ollama ls
```

- **Run a Model:** Loads the model from storage into your working memory (RAM/VRAM) and opens an interactive chat session. If the model isn't downloaded yet, this command will automatically pull it first.

```
ollama run llama3.2:1b
```

- **Delete a Model:** Removes the model weights from your system to free up storage space.

```
ollama rm llama3.2:1b
```

2.2 In-Session (Slash) Commands

Once you are inside an active `ollama run` chat session, you can use slash commands to inspect and modify the model's behavior in real-time.

- **Exit the Session:**

```
/bye
```

- **Inspect Model Details:** View the architecture, parameters, and capabilities (e.g., whether it supports vision or tool calling).

```
/show info  
/show parameters
```

- **Modify Behavior on the Fly:** You can change generation parameters or give the model a new persona instantly.

```
# Make the model more creative  
/set parameter temperature 0.8  
  
# Give the model a specific role  
/set system "You are an expert Python code reviewer. Be  
concise."
```

2.3 Multimodal Execution (Vision)

If you are running a model with vision capabilities (like LLaVA), you can pass images directly into the chat by providing the file path.

```
# Inside a vision model session:  
> Summarize this image: C:\Users\Ahmad\Pictures\diagram.png
```

3 Part 3: Interacting via the Python Library

3.1 Installation and Basic Generation

To use Ollama programmatically, install the official Python wrapper:

```
pip install ollama
```

The `generate` method is used for stateless, single-turn interactions. It does not remember past context.

```
import ollama

# Basic generation
response = ollama.generate(
    model='llama3.2:1b',
    prompt='Why is the sky blue?'
)
print(response['response'])

# Streaming generation (returns chunks as they generate)
stream = ollama.generate(
    model='llama3.2:1b',
    prompt='Write a short poem about AI.',
    stream=True
)
for chunk in stream:
    print(chunk['response'], end='', flush=True)
```

3.2 Customizing Generation Parameters

You can pass system instructions and fine-tune model parameters (like temperature) directly inside the `generate` method using the `options` dictionary.

```
response = ollama.generate(
    model='llama3.2:1b',
    prompt='Explain black holes.',
    system='You are a funny assistant. Answer in pirate speak.',
    options={
        'temperature': 0.8,
        'top_p': 0.9
    }
)
print(response['response'])
```

3.3 Maintaining Context with the Chat Method

For conversational applications, use the `chat` method. Instead of a single string prompt, you pass a history of messages defining the role and content.

```
messages = [
```

```

    {'role': 'system', 'content': 'You are a helpful math tutor.'
    },
    {'role': 'user', 'content': 'What is 5+5?'},
    {'role': 'assistant', 'content': 'The answer is 10.'},
    {'role': 'user', 'content': 'Multiply that by 4.'}
]

response = ollama.chat(
    model='llama3.2:1b',
    messages=messages
)
print(response['message']['content']) # Output: 40

```

3.4 Multimodal Input (Vision via Python)

To pass images programmatically to a vision model (e.g., LLaVA or Qwen-VL), you must first encode the image into a Base64 string.

```

import base64
import ollama

# 1. Read and encode the image
with open('diagram.png', 'rb') as img_file:
    img_b64 = base64.b64encode(img_file.read()).decode('utf-8')

# 2. Pass the Base64 string in the 'images' array
response = ollama.generate(
    model='qwen2-vl',
    prompt='Summarize the data in this chart.',
    images=[img_b64]
)
print(response['response'])

```

4 Part 4: Advanced Capabilities - Tool Calling & Modelfiles

4.1 Tool Calling (Function Calling)

Tool calling allows an LLM to interact with external tools and databases by returning structured arguments rather than standard text.

4.1.1 The 4-Step Tool Calling Workflow

1. **Define the Tool:** Write the Python function (e.g., database lookup, weather API call).
2. **Define the Tool Schema:** Create a JSON schema defining the tool name, description, required parameters, and data types.

3. **Invoke the Model with Tools:** Pass the schema array to `ollama.chat(..., tools=[schema])`.
4. **Execute and Return:** If the model chooses to call a tool, parse its returned arguments, execute the local function, append the result to the message history, and re-query the model for the final response.

4.1.2 Python Implementation Example

```
import ollama

# 1. Python Function
def get_stock_price(symbol: str) -> str:
    # Simulated DB/API lookup
    return "150.00_USD"

# 2. Tool Schema
tool_schema = {
    'type': 'function',
    'function': {
        'name': 'get_stock_price',
        'description': 'Get the current stock price for a given ticker symbol',
        'parameters': {
            'type': 'object',
            'properties': {
                'symbol': {'type': 'string', 'description': 'Stock ticker symbol (e.g. AAPL)'}
            },
            'required': ['symbol']
        }
    }
}

# 3. Model Request with Tools
messages = [{'role': 'user', 'content': 'What is the stock price of AAPL?'}]
response = ollama.chat(
    model='qwen2.5:7b',
    messages=messages,
    tools=[tool_schema]
)

# 4. Process Tool Call Response
if response['message'].get('tool_calls'):
    for tool in response['message']['tool_calls']:
        if tool['function']['name'] == 'get_stock_price':
            arg_symbol = tool['function']['arguments']['symbol']
            result = get_stock_price(arg_symbol)

    # Append execution output back to chat context
```

```

messages.append(response['message'])
messages.append({'role': 'tool', 'content': result})

# Final completion with tool data
final_response = ollama.chat(model='qwen2.5:7b',
    messages=messages)
print(final_response['message']['content'])

```

4.2 Custom Models using Modelfiles

A Modelfile creates a customized, reusable model instance on top of an existing base model without changing underlying weights.

4.2.1 Key Modelfile Directives

- FROM: Defines the base model (e.g., FROM llama3.2:1b).
- PARAMETER: Sets default execution parameters (e.g., PARAMETER temperature 0.2).
- SYSTEM: Assigns a persistent system prompt or persona.

4.2.2 Building a Custom Sentiment Analysis Model

Create a plain text file named Modelfile with the following content:

```

FROM llama3.2:1b

# Set low creativity for deterministic output
PARAMETER temperature 0.0

# Enforce strict system instructions
SYSTEM """
You are an enterprise sentiment analysis engine.
Analyze the input text and respond ONLY with a JSON object
containing two keys:
- sentiment: "positive", "negative", or "neutral"
- confidence: a float between 0.0 and 1.0
Do NOT include any extra explanation or Markdown styling.
"""

```

Build and test your custom model using the CLI:

```

# Build the custom model named 'sentiment-analyzer'
ollama create sentiment-analyzer -f ./Modelfile

# Run and test the new model
ollama run sentiment-analyzer "I absolutely love this new
    framework!"
# Output: {"sentiment": "positive", "confidence": 0.98}

```

5 Part 5: Architecture, LangChain & Ollama Cloud

5.1 The Underlying REST API

Every interaction with Ollama—whether through the CLI or Python—is fundamentally a REST API call to a local server running at `http://localhost:11434`.

You can interact with the API directly using standard HTTP requests:

```
import requests
import json

url = "http://localhost:11434/api/generate"
payload = {
    "model": "llama3.2:1b",
    "prompt": "Explain REST APIs.",
    "stream": False
}

response = requests.post(url, json=payload)
print(response.json()['response'])
```

5.2 Orchestration with LangChain

For production applications (like RAG systems), LangChain acts as an orchestrator. It manages complex pipelines (PDF parsing → chunking → embeddings → vector storage → LLM generation).

You must install the specific LangChain integration package:

```
pip install -U langchain-ollama
```

5.2.1 LangChain Code Example

```
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_core.prompts import ChatPromptTemplate

# 1. Chat/Generation
llm = ChatOllama(model="llama3.1")
prompt = ChatPromptTemplate.from_template("Question: {question} \n\n Answer: Let's think step by step.")
chain = prompt | llm

response = chain.invoke({"question": "What is Quantum Entanglement?"})
print(response.content)

# 2. Generating Embeddings (for Vector Databases)
embedder = OllamaEmbeddings(model="nomic-embed-text")
vector = embedder.embed_query("What is LangChain?")
print(f"Embedding dimensions: {len(vector)}")
```


5.3 Bypassing Hardware Limits with Ollama Cloud

Running massive models (e.g., 100+ billion parameters) locally requires enterprise-grade hardware. **Ollama Cloud** solves this by offloading computation to Ollama’s remote GPUs while keeping the developer experience identical.

5.3.1 Cloud Workflow

1. **Authenticate:** Create an account at ollama.com, then run `ollama signin` in your terminal and paste the generated URL to link your machine.
2. **Run Cloud Models:** Use the exact same CLI commands or Python code to target cloud-specific models (e.g., `ollama run deepseek-v3:671b`).

Note: While Ollama Cloud provides massive compute power, it requires sending your prompt data over the internet, trading away the strict privacy benefits of local execution.